

TradeDB EBRT v2 Research Report

Executive summary

The repo already contains an EBRT-shaped system: evidence ingestion and storage, attention-weighted belief updates, recommendation generation with traces, a portfolio-level Trust Gate (QualityReport) and an “Operational Readiness + Recovery” plane that can backfill data and replay downstream nodes. This is visible in the single FastAPI app that exposes `/api/kline/{symbol}`, `/api/state/{symbol}`, `/api/explain/{symbol}`, and the readiness endpoints `/api/readiness-grid`, `/api/readiness/replay-plan`, `/api/readiness/history`, `/api/readiness/backfill`, `/api/readiness/replay`. ¹

Your current symbol page fails the user goal (“Can I hold/execute a position in this stock?”) mainly for three reasons:

1) Kline data correctness is likely broken by a storage-path mismatch + adjustment policy mismatch.

`DataGateway` loads kline from `data_root/kline/...` ² while `KlineFetcher` writes to `data_root/market/kline/...` ³ — so the kline endpoint can end up empty/degraded, yielding “weird” candles. Separately, the fetcher defaults to `adjust="hfq"` (后复权) ³, which can make displayed prices diverge from what users see on 东方财富 if they’re comparing against 不复权/前复权.

2) The kline endpoint claims indicators but does not compute them, and the UI chart is too minimal for judgement.

`ExplanationService.build_kline_context()` says it “computes indicators” but `_read_ohlcv()` returns only OHLCV rows and a meta `{rows,start,end}` dictionary. ⁴ Meanwhile the frontend type expects `indicators?: Record<string, number>` in `KlineResponse`, i.e. actual indicator values, not a metadata blob. ⁵ The current SVG chart component cannot replicate 东方财富’s “K线+均线+指标面板” experience, so users lack the standard visual anchors (MA, RSI/KDJ/MACD, volume panels, etc.).

3) The system has the machinery to produce “reasons”, but the symbol page is not using it and WorldState has empty evidence lists.

The decision layer already writes `Recommendation` and `RecommendationTrace`, and can generate “reasons” from the top attention scores (Evidence → Attention → Reasons) via `build_reasons()`. However, the explain stack (`DecisionExplanation`) will fall back to generic regime labels when no `supporting_signals`/`raw_reasons` are supplied. That produces shallow explanations like “market:TRENDING_UP” instead of concrete, user-salient reasons such as “MA5 crossed MA20”, “KDJ golden cross”, “last 5 days drawdown + volume spike”, “Trust gate constrained due to missing features”.

A viable v2 should therefore: (a) fix kline storage + expose 复权 controls; (b) return full indicator series and adopt an 东方财富-like layout; (c) wire existing EBRT traces (Attention/Reasons/Trust vector) into a structured “Why/Why-not/What-changes” explanation that answers **Hold / Add / Reduce / No-action** with quantified trust and explicit constraints; (d) integrate “freshness + recovery” into the symbol page so users can repair and recompute before acting.

Repository and document coverage

Enabled connectors: **GitHub** (only). ⁶

Files read first (to anchor EBRT → code mapping and the UI/data problems):

- Backend API surface + readiness: `trade_web/backend/app.py`, `trade_web/backend/readiness.py`
- Symbol explain/decision stack: `trade_py/services/explanation_service.py`, `trade_py/services/state_service.py`, `trade_py/services/decision_service.py`
- WorldState + explanation contract: `trade_py/decision/world_state.py`, `trade_py/decision/explanation.py`
- EBRT belief update + attention: `trade_py/belief/update.py`, `trade_py/belief/attention.py`, `trade_py/belief/__init__.py`, `trade_py/engine/__init__.py`
- Trust Gate / QualityReport: `trade_py/evaluation/trust.py`, `trade_py/evaluation/service.py`
- Kline ingestion + store: `trade_py/data/access/gateway.py`, `trade_py/data/market/kline/akshare.py`, `trade_py/utils/data_inspector.py`
- Recommendation + trace + reasons: `trade_py/decision/__init__.py`, `trade_py/decision/explain.py`, `trade_py/decision/rank.py`
- Frontend symbol page: `trade_web/frontend/src/pages/SymbolPage.tsx`, `trade_web/frontend/src/components/SymbolChart.tsx`, `trade_web/frontend/src/components/SymbolDecisionHeader.tsx`, `trade_web/frontend/src/components/ExplanationRail.tsx`, `trade_web/frontend/src/components/SymbolFreshnessBanner.tsx`, `trade_web/frontend/src/lib/api.ts`

EBRT_02.pdf note: the repo exposes EBRT concepts directly in code (attention formula references “EBRT.pdf §Algorithms” in `belief/attention.py`). ⁷ In this tooling run, the PDF itself was not reliably citeable as text; conclusions below are therefore based on the executable spec already embedded in the code paths above.

EBRT_02 key algorithms and claims as implemented in code

The codebase expresses EBRT as a layered pipeline:

• Evidence → Attention → Belief update (daily).

`BeliefEngine` (invoked by `engine.update_belief`) reads prior belief, collects evidence (Gold sentiment, market events, articles), computes attention logits + softmax weights, detects conflict, computes an adaptive gain η , and applies a residual update.

The update equation is explicitly documented and implemented:

• Belief residual update:

$$b_{t+1,s} = (1 - \lambda) \cdot b_{t,s} + \eta_{t,s} \cdot \sum_i w_{t,s,i} \cdot \Delta(e_{t,s,i}) \quad 8$$

• Adaptive gain:

$$\eta = \text{clip}(\eta_0 \cdot \text{TrustGate}_t \cdot (1 - \text{Drift}_t) \cdot \bar{r}, 0, 1)$$

- Attention logits combine compatibility, magnitude, reliability, novelty, penalties, and a conflict term, then softmax: ⁷

- **Recommendation generation + trace (daily).**

`produce_recommendations()` writes freshness, ranks symbols using a 3-factor composite score (belief_mu, window_score, event_kg_score) and emits both `Recommendation` and `RecommendationTrace`.

It also generates human-readable “reasons” from `AttentionScore` + `Evidence` (top-N).

- **Trust is not “one confidence score”; it is a gate + vector.**

The daily evaluator writes a **7-component trust vector** and collapses it to a scalar **T*** via a sigmoid-weighted sum.

The trust vector components are explicitly defined (freshness, evidence reliability, model IC proxy, calibration via Brier, drift via MMD proxy, ops stability, explain coverage). ⁹

This matches standard references: Brier score originates from Brier (1950) and is a proper score for probabilistic forecasts. ¹⁰ Drift via MMD aligns with the “maximum mean discrepancy” family of two-sample tests (Gretton et al., JMLR 2012). ¹¹

- **Operational readiness + recovery is a first-class plane.**

The readiness system defines a dataset catalogue and, for each dataset, a downstream replay closure. For `kline` this closure includes

`window_score → belief_update → recommend → evaluate_daily`. ¹²

This matters for your earlier concern: **the system already encodes “backfill first, then recompute trust/recommendation”** as the default downstream chain for repairs.

Mapping EBRT components to concrete code paths

Evidence layer

- Storage + CRUD for EBRT tables is centralised in `TradeDB` and `EBRTCRUDMixin` (`Evidence`, `BeliefState`, `AttentionScore`, `BeliefTransition`, `Recommendation`, `RecommendationTrace`).
- Evidence is produced in `BeliefEngine._build_evidence_list()` from sentiment gold and market events, and upserted into the DB. ¹³

Belief layer

- Attention: `trade_py/belief/attention.py` defines the features that drive evidence weighting and implements softmax. ⁷
- Belief update: `trade_py/belief/update.py` defines the residual update and multi-horizon approximation. ⁸
- Orchestration: `trade_py/belief/__init__.py` runs daily updates and records `AttentionScore` and `BeliefTransition`. ¹³
- Entry point: `trade_py/engine/__init__.py.update_belief()` is the canonical job hook. ¹⁴

Recommendation layer

- Ranking: `trade_py/decision/rank.py` computes the composite score and maps it to buy/watch/avoid + conviction. ¹⁵
- Writing: `trade_py/decision/__init__.py.produce_recommendations()` writes `FreshnessStatus`, `Recommendation`, and `RecommendationTrace`, and creates reasons from the top attention evidence.

Trust layer

There are **two trust notions** in the repo:

1) Portfolio/system trust gate (QualityReport T*)

Implemented in `trade_py/evaluation/trust.py`: computes trust vector, scalar T*, writes QualityReport, and updates source reliabilities.

2) Per-symbol prediction trust (feature coverage + freshness)

Implemented in `trade_py/trust/compute.py`: computes a per-prediction trust score from missing features, defaults usage, and data lag (with a built-in floor). ¹⁶

This is what the inference stack returns to the symbol explain route (`ExplanationService._get_trust`).

WorldState and decision layer

- `WorldState` is a deterministic regime labeller that is meant to be the sole source for downstream decision/explain consumers. ¹⁷
- `StateService` builds `WorldState` by reading signals, belief state, events, and a lightweight freshness score (derived from `sync_state` and snapshot metadata). ¹⁸
- `DecisionService` derives scenario summary + action decision (WATCH/PROBE/ADD/REDUCE) and can incorporate inference `model_score/model_risk` if available.

Readiness + recovery plane

- Dataset catalogue and downstream closure: `trade_web/backend/readiness.py` (`DATASET_CATALOG`, `DOWNSTREAM_JOB_MAP`). ¹²
- Execution: recovery actions are stored in `readiness_recovery_actions` and executed as a step list of jobs using `run_node(job_name, date_from, date_to)`.
- API: wired in `trade_web/backend/app.py` (`/api/readiness-grid`, `/api/readiness/replay-plan`, `/api/readiness/history`, `/api/readiness/backfill`, `/api/readiness/replay`). ¹

Mismatches and missing interfaces blocking the user goal

Kline correctness and “weird candles”

- **Storage-path mismatch is the most likely immediate bug.**

`DataGateway._load_kline_local()` reads `data_root/kline/YYYY-MM/*.parquet`. ²

`KlineFetcher` writes to `data_root/market/kline/YYYY-MM/*.parquet`. ³

Net effect: `/api/kline/{symbol}` can silently return `[]` OHLCV, which makes the frontend chart scale oddly or look empty.

The repo already contains a resolver in `data_inspector.py` that tries both `market/kline` and `kline`; `DataGateway` should do the same.

- **复权 policy mismatch will produce “data not matching 东方财富”.**

The fetcher defaults to `"hfq"` (后复权). ³

AKShare documents that `adjust=""` returns 不复权, `qfq` returns 前复权, and `hfq` returns 后复权. ¹⁹

If 东方财富 is showing 不复权 or 前复权 (common for K线 technical viewing), then your displayed close prices will look “wrong” under 后复权. (Even if the trend is correct, the absolute level can differ.)

Kline endpoint does not deliver “decision-grade” indicators

- `build_kline_context()` claims it “computes indicators” but returns only OHLCV rows + meta. ⁴
- Frontend expects `indicators` but will not receive MA/KDJ/MACD series from the backend. ⁵
- WorldState technical regime logic currently uses RSI + MACD signal only; `TechnicalState` includes `kdj_cross` but it is never inferred/filled.
This directly conflicts with your requirement that reasons include “KDJ 金叉”等.

Explanation is too shallow despite existing EBRT trace machinery

- The repo already generates reasons from attention + evidence (`build_reasons`) and stores traces in `RecommendationTrace`.
- But the symbol page explain path (`/api/explain/{symbol}`) → `ExplanationService.explain`) does not fetch those stored reasons/trace and does not compute technical signals; it therefore falls back to regime label strings.
- `WorldState.supporting_signals/opposing_signals` are empty because `StateService.build()` does not populate them and `build_world_state()` only uses them if passed.

Recovery semantics for sparse/internal gaps

- `DataGateway.get_kline()` only checks whether the local max date reaches `target_end`; it does not detect internal missing trading days, so **non-contiguous gaps** will not be repaired. ²
- The readiness grid also largely reasons about “coverage to a day” not “continuity within a window”. ¹²
This means your earlier concern stands: sparse gaps can exist without triggering a repair plan.

Trust confusion: per-symbol trust vs global trust gate

- The explain route reconstructs trust from inference (`compute_prediction_trust`) which is a feature-coverage/freshness heuristic.
- The belief engine uses the **QualityReport trust_scalar** (T*) as TrustGate to scale learning rate.
- The UI needs both, but today it mostly surfaces “low_feature_coverage” and hides the “system-level permission to act” concept that EBRT intends.

Next-version design that converges on the user goal

Product goal restatement

User question: “Can I hold/execute a position in this stock?”

EBRT answer form: *Action + Confidence + Trust + Reasons (auditable evidence chain) + Invalidators / next triggers + Data readiness constraints & recovery controls.*

In practical UI terms: the symbol page must become a **decision cockpit** that merges (1) 东方财富-style charting affordances and (2) EBRT explainability + operational gating.

Proposed UI prototype sketch

ASCII wireframe (desktop):

[603083.SH 剑桥科技] 最新价 115.33 涨跌 -0.34(-0.29%) 成交量 35.07万 今开 118.00 最高 122.80 最低 114.46 昨收 115.67 换手 12.73% 市值... 复权: (不复权 ▾) 周期: 日K 周K 月K 5m 15m ... 指标: MA BOLL ... 数据: □截至 2026-03-20 ⚠若缺失: [一键修复并重算] [查看就绪性]	
K线主图 (蜡烛 + MA5/10/20/60 + 事件标记) REDUCE Medium (HIGH line) ended..." 成交量面板 ... □] 指标面板 tabs: RSI KDJ MACD ... recompute]	决策卡 (Answer) Action: HOLD / ADD / Confidence: Trust: 0.72 Thesis (1 "Trend up, pullback Constraints: none / [I hold position [Refresh &
Why (Top reasons, evidence-backed) 1) MA5 crossed above MA20 (2026-03-18), price above MA20 (bullish) 2) KDJ golden cross confirmed, J rising from oversold region 3) Past 5d drawdown -8% with vol spike 1.8×, now stabilising (mean reversion) 4) Event KG: positive propagation score 0.42 (industry upgrade news...) 5) TrustGate OK: T* 0.76, drift low, calibration acceptable Why-not / Risks:	

```

| - High uncertainty (sigma=0.41) → smaller sizing / wait for
confirmation
| - Stale fundamentals coverage → treat long-term thesis with
caution
|
|
| What changes my mind (Invalidators /
triggers):
| - Close < MA20 for 2 sessions; MACD turns negative; new negative
event...

```

Key design choices:

- **Default: do not show belief μ/σ** (your preference). Offer an “Advanced” toggle that reveals belief overlay and attention details. This matches EBRT’s “auditable trace” without overwhelming most users.
- Reasons are grouped into **Technical / Price Action / Events / Model / Data Quality** and each item is machine-generated with exact dates/values.

Kline and 复权 design

Add an explicit `adjust` control with three states:

- 不复权 (`adjust=""`) for “what the market traded”
- 前复权 (`adjust="qfq"`) for indicator continuity (common for technical analysis)
- 后复权 (`adjust="hfq"`) for total-return style long-term context

AKShare supports exactly these semantics (`""`, `qfq`, `hfq`). ¹⁹

Implementation should stamp the chart with the chosen adjustment so screenshots are explainable.

Explanation design: from EBRT trace to user-salient reasons

Use the existing four-layer `DecisionExplanation` contract, but **fill it with concrete facts**:

- Layer 1 (Input quality):
 - show per-symbol prediction trust (feature coverage/freshness) ¹⁶
 - show system TrustGate T^* and its vector components (fresh/evidence/model/calib/drift/ops/explain) ⁹
 - if constrained, show exactly which dataset/blocker caused it (already supported by `SymbolFreshnessBanner`).
- Layer 2 (State rationale):

Expand `WorldState.technical_state` beyond RSI into MA structure, MACD and KDJ cross status (currently missing). ¹⁷
- Layer 3 (Decision):

Preserve `ActionDecision` (ADD/PROBE/REDUCE/NO_ACTION) and optionally parameterise by `has_position`.

- Layer 4 (Counter-argument):
Make invalidators and “next triggers” numeric and tied to indicators (thresholds) rather than vague strings.

Recovery flow that handles sparse gaps and “recompute after repair”

The repo already has a recovery closure for kline that ends in `evaluate_daily` (writes QualityReport).

v2 should add **symbol-page-triggered recovery** with continuity checks.

Mermaid sequence for “symbol open → repair → recompute”:

```
sequenceDiagram
    participant UI as Symbol Page
    participant API as FastAPI
    participant DG as DataGateway
    participant R as Readiness/Recovery
    participant ENG as Engine jobs
    participant DB as TradeDB

    UI->>API: GET /api/kline/{symbol}?adjust=qfq&days=120
    API->>DG: get_kline(symbol, end_date=as_of)
    DG-->>API: OHLCV + report(degraded? gaps?)
    alt gaps or stale detected
        API-->>UI: payload + Banner("Constrained", repair suggested)
        UI->>API: POST /api/readiness/backfill {dataset:"kline", date_from,
        date_to, mode:"data_plus_downstream"}
        API->>R: execute_recovery_action (kline_update -> window_score ->
        belief_update -> recommend -> evaluate_daily)
        R->>ENG: run_node(...) in range
        ENG->>DB: write Evidence/Belief/Recommendation/QualityReport
        R-->>API: action_id + steps
        UI->>API: GET /api/readiness/history?dataset=kline&date=...
        UI->>API: GET /api/kline/{symbol} (refresh)
    else no gaps
        API-->>UI: full chart + reasons
    end
    end
```

To handle **non-contiguous internal gaps**, v2 should:

- compute expected trading days (using `trading_calendar` table already used elsewhere in the readiness system) and compare to local OHLCV dates within a rolling window (e.g., last 120 bars);
- if gaps exist, repair by fetching the **convex hull** `[min_missing, max_missing]` (PO pragmatic), and later optimise into disjoint segments (P1).

Data contracts: current vs proposed payloads

`/api/kline/{symbol}`

Aspect	Current payload	Proposed v2 payload
OHLCV	list of dicts (may be empty due to path mismatch)	always resolved from correct dir; includes <code>adjust</code> applied; includes <code>data_report</code>
indicators	meta-only <code>{rows, start, end}</code> (not indicators)	series: MA5/10/20/60, RSI(6/12/24), KDJ(K/D/J + cross), MACD(DIF/DEA/HIST), ATR, etc.
explanation	summary dict only ¹	embed TopReasons (technical + event + model + data) with values/dates
adjustment	fixed by storage defaults (<code>hfq</code>) ³	explicit query param <code>adjust ∈ {"none", "qfq", "hfq"}</code> with UI control

`/api/explain/{symbol}`

Aspect	Current	Proposed v2
evidence_for/against	often generic fallback because ws signals empty	merge (a) Recommendation reasons from Attention/Evidence and (b) technical facts computed from kline
trust	mainly per-prediction trust	show both: per-symbol trust + system TrustGate (QualityReport trust_scalar + vector)
has_position	not parameterised	add query <code>?has_position=1</code> to switch HOLD/REDUCE logic

Sample JSON schemas

New recovery request (symbol-driven but still uses existing backfill endpoint)

```
{
  "type": "object",
  "required": ["dataset", "date_from", "date_to", "mode"],
  "properties": {
    "dataset": { "type": "string", "enum": ["kline", "fund_flow", "fundamental", "events", "signals", "belief_state", "recommendation"] },
    "date_from": { "type": "string", "format": "date" },
    "date_to": { "type": "string", "format": "date" },
    "mode": { "type": "string", "enum": ["data_only", "data_plus_downstream", "full_replay"] },
    "symbol": { "type": "string", "description": "Optional optimisation hint; backend may ignore in P0." },
    "reason": { "type": "string", "description": "UI-provided reason e.g. 'symbol_page_refresh'." }
  }
}
```

```
}  
}
```

Recovery result with steps (already close to current)

```
{  
  "type": "object",  
  "required": ["accepted", "action_id", "plan"],  
  "properties": {  
    "accepted": { "type": "boolean" },  
    "action_id": { "type": "integer" },  
    "plan": {  
      "type": "object",  
      "properties": {  
        "dataset": { "type": "string" },  
        "job_name": { "type": ["string", "null"] },  
        "recommended_mode": { "type": "string" },  
        "full_chain": {  
          "type": "array",  
          "items": { "type": "object", "properties": { "job_name": { "type":  
"string" }, "stage": { "type": ["string", "null"] } } }  
        }  
      }  
    },  
    "result": {  
      "type": "object",  
      "properties": {  
        "steps": {  
          "type": "array",  
          "items": {  
            "type": "object",  
            "properties": {  
              "job_name": { "type": ["string", "null"] },  
              "status": { "type": ["string", "null"], "enum": ["ok",  
"error", "running", "queued", null] },  
              "summary": { "type": ["string", "null"] },  
              "duration_ms": { "type": ["integer", "null"] }  
            }  
          }  
        },  
        "duration_ms": { "type": ["integer", "null"] }  
      }  
    }  
  }  
}
```

Exact code changes and implementation plan

P0 changes

Backend (data correctness + indicators + reasons):

- 1) **Fix kline local-read path resolution** - File: `trade_py/data/access/gateway.py`
 - Change `_load_kline_local()` to resolve both `market/kline` and `kline` like `data_inspector` does.
 - Add unit test that writes a parquet to `tmp/market/kline/2026-03/600000_SH.parquet` and asserts `DataGateway.get_kline()` returns rows.
- 2) **Expose adjust parameter end-to-end** - File: `trade_web/backend/app.py`: extend `/api/kline/{symbol}` signature to accept `adjust` query parameter and pass it to `build_kline_context`.¹
 - File: `trade_py/services/explanation_service.py`: add `adjust` param to `_read_ohlcv()` and/or read path keyed by `adjust`.⁴
 - Default recommendation: **display uses** `adjust=none`, indicator computations can optionally use `qfq` (user selectable).
- 3) **Actually compute indicator series in** `build_kline_context` - File: `trade_py/services/explanation_service.py`
 - Replace current `indicators_meta` with a structure like: - `ma`: arrays per bar for MA5/10/20/60 - `rsi`: arrays - `kdj`: arrays + cross events - `macd`: arrays
 - Return these under `indicators` to match frontend expectations.
- 4) **Wire stored EBRT reasons/trace into** `/api/explain` - File: `trade_py/services/explanation_service.py`
 - When building explanation, read: - `Recommendation` row for `as_of_date` (or latest $\leq$ date) - `reasons_json` and/or call `build_reasons()` if missing (it uses Attention/Evidence).
 - Pass `raw_reasons` into `build_explanation()` so evidence becomes concrete.
- 5) **Add KDJ + MA crossover reasons** - File: `trade_py/decision/world_state.py` or a new `trade_py/analysis/technicals.py`
 - Populate `supporting_signals/opposing_signals` with computed facts (date/value) so `DecisionExplanation` can show them.
 - Keep belief μ/σ out of the default UI, but retain for "Advanced".

Frontend (东方财富-style usability):

- 6) **Replace SymbolChart with a candlestick chart library + indicator panels** - Files:
 - `trade_web/frontend/src/components/SymbolChart.tsx` (replace)²⁰
 - `trade_web/frontend/src/pages/SymbolPage.tsx` integrate controls²¹
 - `trade_web/frontend/src/components/SymbolDecisionHeader.tsx` redesign header to match 东方财富 info density²²
 - Add UI controls: timeframe, adjust selector, indicator tab switcher, full-screen.
- 7) **Reasons panel redesign** - File: `trade_web/frontend/src/components/ExplanationRail.tsx`
 - Render grouped reasons with values and timestamps; show "Why / Why-not / Invalidators / Next triggers".

8) **Symbol-level freshness + one-click recovery** - Reuse `SymbolFreshnessBanner` but add a prominent "Repair & recompute" CTA wired to `postReadinessBackfill`.

P1 changes

- **Internal-gap detection** in `DataGateway.get_kline()` using trading calendar continuity; store missing ranges and trigger range backfill.
- **Unify trust presentation**: show both per-symbol trust and system TrustGate T*, with a clearly labelled breakdown (avoid conflating them).
- **Add `has_position` parameter** to `/api/explain` and to the frontend toggle (hold vs no position).

P2 changes

- **Separate storage by adjustment** (e.g., `market/kline_qfq`, `market/kline_none`) to avoid mixed datasets and to make backfills deterministic.
- **Symbol-scoped recovery mode** that replays just the symbol's derived artifacts (signals/belief/recommend) instead of a whole-universe recompute.

Effort and milestones

Milestone A (P0 data + API correctness): **Medium**

- Fix path resolver + add adjust param + compute indicators + wire reasons.

Milestone B (P0 UI redesign for 东方财富-style charting): **High**

- Candlestick + overlays + indicator panels + header redesign.

Milestone C (P1 robustness): **Medium**

- Gap continuity checks + trust unification + `has_position` flow.

Verification checklist (must-pass)

Backend API checks (manual + automated):

- `GET /api/readiness-grid?days=30` returns payload with datasets/rows.
- `GET /api/readiness/history?dataset=kline&date=YYYY-MM-DD` returns items list.
- `GET /api/readiness/replay-plan?dataset=kline&date_from=...&date_to=...` returns `full_chain`.
- `POST /api/readiness/backfill` returns `accepted` and spawns steps that include `evaluate_daily` for kline repairs.
- `GET /api/kline/{symbol}?adjust=none` returns non-empty OHLCV for a known symbol and indicator series arrays.
- `GET /api/explain/{symbol}` returns evidence items populated from reasons/attention rather than only regime fallbacks.
- `GET /api/state/{symbol}` includes technical rationale and, after P0/P1, populated signals.

Unit tests to add (pytest):

- `test_kline_gateway_reads_market_dir()` for the path fix.
- `test_kline_adjust_param_pass_through()` verifying `adjust` affects returned prices (smoke). ¹⁹

- `test_explain_includes_attention_reasons()` verifying `raw_reasons` appear as `evidence_for` in `DecisionExplanation`.

Risks, assumptions, and open questions

- **复权 policy decision:** which adjustment should be the default for *display vs feature computation*? AKShare supports all three, but mixing them in storage can corrupt user trust unless clearly labelled. ¹⁹
- **Authoritative data cadence:** `DataGateway` uses a CST 16:00 cutoff and skips weekends but not explicit holidays. If users compare to 东方财富 on a holiday week, perceived mismatches may persist unless the trading calendar is consulted.
- **Replay cost:** readiness closures currently replay whole jobs across date ranges (often universe-wide). Symbol-scoped replays may be needed for interactive UX.
- **Explainability language:** reasons are partially Chinese in `decision/explain.py`; the UI should support bilingual or consistent CN output if your audience expects 东方财富-like CN explanations.
- **Trust calibration:** the repo uses Brier and an MMD proxy for drift. These are defensible, but you may want temperature scaling for calibration of probabilistic models (Guo et al., 2017) if you rely on model probabilities. ²³

Final P0/P1/P2 checklist for a coding agent

P0 checklist

- [] Fix kline directory mismatch: `DataGateway` reads from `market/kline` (and fallback to `kline`).
- [] Add `adjust` query param to `/api/kline/{symbol}` and thread it into data fetch.
- [] Compute and return indicator series (MA/RSI/KDJ/MACD) instead of meta-only `indicators`.
- [] Integrate `Recommendation` / `AttentionScore` reasons into `/api/explain/{symbol}` via `raw_reasons`.
- [] Replace SVG chart with 东方财富-like candlestick + indicator panels; add `adjust` selector and indicator tabs.
- [] On symbol page, show one clear Answer card: Action + Confidence + Trust + Thesis; hide belief by default.

P1 checklist

- [] Detect internal gaps in kline continuity and trigger convex-hull range repair.
- [] Unify trust display: per-symbol trust + system TrustGate T^* (vector + scalar).
- [] Add `has_position` toggle in UI and `?has_position=1` in `/api/explain`.

P2 checklist

- [] Store multiple adjustment modes separately (none/qfq/hfq) to avoid ambiguity. ¹⁹
- [] Add symbol-scoped recovery mode to reduce replay cost while preserving EBRT closure semantics.

¹ `app.py`

https://github.com/Hwloser/trade/blob/main/trade_web/backend/app.py

- 2 **gateway.py**
https://github.com/Hwloser/trade/blob/main/trade_py/data/access/gateway.py
- 3 **akshare.py**
https://github.com/Hwloser/trade/blob/main/trade_py/data/market/kline/akshare.py
- 4 **explanation_service.py**
https://github.com/Hwloser/trade/blob/main/trade_py/services/explanation_service.py
- 5 **api.ts**
https://github.com/Hwloser/trade/blob/main/trade_web/frontend/src/lib/api.ts
- 6 **README.md**
<https://github.com/Hwloser/trade/blob/main/README.md>
- 7 **attention.py**
https://github.com/Hwloser/trade/blob/main/trade_py/belief/attention.py
- 8 **update.py**
https://github.com/Hwloser/trade/blob/main/trade_py/belief/update.py
- 9 **trust.py**
https://github.com/Hwloser/trade/blob/main/trade_py/evaluation/trust.py
- 10 **<https://ui.adsabs.harvard.edu/abs/1950MWRv...78....1B/abstract>**
<https://ui.adsabs.harvard.edu/abs/1950MWRv...78....1B/abstract>
- 11 **<https://jmlr.csail.mit.edu/papers/v13/gretton12a.html>**
<https://jmlr.csail.mit.edu/papers/v13/gretton12a.html>
- 12 **readiness.py**
https://github.com/Hwloser/trade/blob/main/trade_web/backend/readiness.py
- 13 **__init__.py**
https://github.com/Hwloser/trade/blob/main/trade_py/belief/__init__.py
- 14 **__init__.py**
https://github.com/Hwloser/trade/blob/main/trade_py/engine/__init__.py
- 15 **rank.py**
https://github.com/Hwloser/trade/blob/main/trade_py/decision/rank.py
- 16 **compute.py**
https://github.com/Hwloser/trade/blob/main/trade_py/trust/compute.py
- 17 **world_state.py**
https://github.com/Hwloser/trade/blob/main/trade_py/decision/world_state.py
- 18 **state_service.py**
https://github.com/Hwloser/trade/blob/main/trade_py/services/state_service.py
- 19 **AKShare 股票数据 — AKShare 1.4.92 documentation**
https://akshare-hh.readthedocs.io/en/latest/data/stock/stock.html?utm_source=chatgpt.com
- 20 **trade_db.py**
https://github.com/Hwloser/trade/blob/main/trade_py/db/trade_db.py
- 21 **ExplanationRail.tsx**
https://github.com/Hwloser/trade/blob/main/trade_web/frontend/src/components/ExplanationRail.tsx
- 22 **SymbolFreshnessBanner.tsx**
https://github.com/Hwloser/trade/blob/main/trade_web/frontend/src/components/SymbolFreshnessBanner.tsx

23 <https://www.shuzhiduo.com/A/amd0eBn1zg/>
<https://www.shuzhiduo.com/A/amd0eBn1zg/>